SOFTWARE REQUIREMENT & SYSTEM DESIGN DOCUMENTATION

Next-Generation WiFi Service Provider Management System (WISP-MS)
Digitization & System Architecture Document

Project Title:	WiFi ISP Digitization & Customer Management System
Prepared For:	WiFi Internet Service Provider Operations
Document Version:	1.0.0 (Comprehensive Release)
Target Stack:	Next.js 14 (Frontend/API), Supabase (PostgreSQL Backend)

1. Executive Summary & Problem Statement

This document outlines the end-to-end Software Development Life Cycle (SDLC), architecture, data modeling, and technical specifications for transforming manual, paper-register-based local Internet Service Provider (ISP) operations into a secure, scalable, and automated cloud platform.

1.1 Operational Pain Points (Current Manual System)

- **Data Loss & Paper Vulnerability:** Manual registers and paper logs are susceptible to physical damage, loss, illegible handwriting, and lack centralized backups.
- **Revenue Leakage & Unchecked Defaulters:** Tracking monthly subscription payments manually across hundreds of active clients leads to missed collections, unrecorded cash transactions, and delayed payment enforcement.
- **Inefficient Complaint Resolution:** Technical faults, line breaks, and customer tickets logged verbally or on loose notes lack status tracking, prioritization, and field technician assignment.
- **Zero Operational Analytics:** Management lacks visibility into real-time active user counts, total bandwidth allocation, churn rates, monthly revenue projections, and outstanding receivables.

1.2 The Proposed Solution

The proposed system—WISP Management System (WISP-MS)—is a web-based administrative dashboard and backend API built using Next.js and Supabase (PostgreSQL). It centralizes customer onboarding, subscription plan management, automated payment/billing tracking, and a real-time ticketing system for technical support.

2. Software Development Life Cycle (SDLC) Methodology

To ensure rapid delivery, high quality, and seamless adaptation to field requirements, the project will follow the Agile Scrum Methodology structured across 6 distinct phases.

Phase	SDLC Stage	Key Deliverables & Activities
Phase 1	Requirements & Discovery	Stakeholder interviews, physical log audit, requirement prioritization, SRS document completion.
Phase 2	System Design & Architecture	Database ERD layout, API schema design, UI/UX high-fidelity wireframing, role permission matrix.
Phase 3	Database & Core Backend Setup	Supabase PostgreSQL setup, Row Level Security (RLS) policies, database triggers, seed scripts.
Phase 4	Frontend & Module Development	Next.js dashboard integration, Customer Management, Billing/Invoice module, Complaint Desk.
Phase 5	Data Migration & Testing	Digitization of historical paper logs into CSV/JSON, bulk data import, unit testing, integration testing.
Phase 6	Deployment & Training	Production deployment on Vercel/Supabase, admin staff training, user manual handover, operational go-live.

3. System Requirements Specification (SRS)

3.1 Functional Requirements

- **FR-1: Customer Onboarding & Profiling:** The system shall allow staff to create, update, deactivate, and view customer records containing Name, Mobile Number, CNIC/ID, Address, Fiber/MAC Address, Router Model, and Installation Date.
- **FR-2: Plan & Package Management:** The system shall store Internet packages with speed metrics (e.g., 10 Mbps, 20 Mbps) and monthly rate pricing.
- **FR-3: Subscription & Status Lifecycle:** The system shall automatically calculate monthly due dates based on activation date and transition delinquent accounts to 'Overdue' or 'Suspended' status.
- **FR-4: Payment Processing & Ledger:** The system shall record full and partial cash/online payments, generate unique digital invoice receipts, and maintain an automated billing history log.

- **FR-5: Complaint & Ticket Desk:** Staff and technicians shall log support issues, assign ticket priorities (Low, Medium, High, Critical), track status (Open, In Progress, Resolved), and log resolution notes.
- **FR-6: Global Search & Analytics Dashboard:** The main dashboard shall display total active users, monthly revenue collected vs. pending, active tickets, and provide multi-parameter quick search (by phone, name, IP).

3.2 Non-Functional Requirements

- **Performance:** Database query response time must be under 200ms for standard customer searches across up to 50,000 active records.
- **Security:** Authentication via Supabase Auth with encrypted tokens, granular Row Level Security (RLS) based on user roles, and HTTPS protocol end-to-end.
- **Usability & Accessibility:** Responsive design optimized for both desktop web browsers (office desk staff) and mobile screens (field technicians).
- **Reliability & Backups:** Automated daily point-in-time PostgreSQL database backups with 99.9% uptime guaranteed via cloud deployment.

4. Technical Architecture & Stack Selection

4.1 Technology Stack

Layer	Technology Selected	Justification & Benefits
Frontend Framework	Next.js 14 (React / App Router)	Server-side rendering (SSR) for fast data load, modern UI capabilities, built-in API routing.
Backend & Database	Supabase (PostgreSQL)	Relational database structure, real-time subscriptions, built-in Auth, instant REST/GraphQL APIs.
UI Component Library	Tailwind CSS + Shadcn UI	Highly responsive, clean aesthetic, fast field-level searching and modular table components.
Authentication & Security	Supabase Auth (JWT + RLS)	Role-based access control (Admin, Receptionist, Technician) with fine-grained row policies.
Deployment & Hosting	Vercel (Frontend) / Supabase Cloud	Zero-downtime serverless infrastructure with auto-scaling capabilities.

5. Database Schema & Data Dictionary (Supabase / PostgreSQL)

The relational database schema is structured into five core tables to guarantee third normal form (3NF) compliance, data integrity, and efficient querying.

1. Table: packages (Internet Plans)

Column Name	Data Type / Key	Description
id	UUID (PK)	Unique identifier
package_name	VARCHAR(50)	Plan name (e.g., Ultra 20M)
speed_mbps	INTEGER	Bandwidth speed in Mbps
monthly_price	DECIMAL(10,2)	Monthly subscription cost
created_at	TIMESTAMP	Record creation timestamp

2. Table: customers (Client Directory)

Column Name	Data Type / Key	Description
id	UUID (PK)	Unique customer ID
full_name	VARCHAR(100)	Customer full name
phone_number	VARCHAR(20)	Primary contact number
cnic_number	VARCHAR(20)	CNIC / ID card number
address	TEXT	Physical connection location
package_id	UUID (FK)	Foreign key referencing packages.id
status	VARCHAR(20)	Status: Active, Overdue, Suspended
installation_date	DATE	Date of connection deployment

3. Table: payments (Financial Ledger)

Column Name	Data Type / Key	Description
id	UUID (PK)	Payment receipt ID
customer_id	UUID (FK)	Foreign key referencing customers.id
amount_paid	DECIMAL(10,2)	Amount collected
payment_date	DATE	Date payment was received

payment_month	VARCHAR(20)	Billing month (e.g., August 2026)
payment_method	VARCHAR(30)	Cash, EasyPaisa, JazzCash, Bank
received_by	VARCHAR(50)	Staff member who collected payment

4. Table: complaints (Support Desk)

Column Name	Data Type / Key	Description
id	UUID (PK)	Ticket ID
customer_id	UUID (FK)	Foreign key referencing customers.id
issue_description	TEXT	Details of technical issue
priority	VARCHAR(20)	Low, Medium, High, Critical
status	VARCHAR(20)	Open, In Progress, Resolved
assigned_technician	VARCHAR(100)	Field staff name
created_at	TIMESTAMP	Ticket creation timestamp

6. Data Migration Strategy (Manual Paper to Cloud)

Migrating historical records from physical paper registers into Supabase will follow a structured 4-step pipeline to maintain complete data accuracy.

- **Step 1: Standardization & Template Creation:** Standardize paper records into a structured Microsoft Excel / CSV template containing mandatory fields: Name, Phone, CNIC, Address, Package Name, and Current Due Amount.
- **Step 2: Data Cleaning & Verification:** Identify duplicate client entries, resolve illegible handwritten contact numbers, and verify active connection statuses against actual physical drop lines.
- **Step 3: Automated Bulk Ingestion:** Execute a custom Python migration script utilizing the Supabase Python SDK to parse cleaned CSV rows, resolve foreign key package IDs, and insert records into the production PostgreSQL database.
- **Step 4: Audit & Verification:** Conduct a 10% random sampling audit comparing generated digital dashboard profiles against original physical registers to verify 100% data alignment.

> 📌 **CRITICAL MIGRATION SAFEGUARD:** *During the 2-week migration window, both physical logging and digital logging will run in parallel (Dual-Entry Mode) to ensure zero disruption to billing cycles.*

7. Testing & Risk Management

7.1 Testing Strategy

- **Unit Testing:** Verify database queries, calculation of monthly arrears, and state transitions for overdue users.
- **Integration Testing:** Test real-time payment recording and check automatic UI updates across multiple logged-in admin sessions.
- **User Acceptance Testing (UAT):** Conduct mock daily operations with ISP receptionists and field technicians to ensure user-friendly interface workflows.

7.2 Risk Assessment Matrix

Identified Risk	Severity	Impact	Mitigation Strategy
Incorrect/Missing Manual Data	High	Inaccurate customer billing	Mandatory fields during CSV cleanup + physical audit verification.
Staff Resistance to Digital System	Medium	Slow system adoption	Intuitive Urdu/English UI labels + 2-day hands-on staff training.
Internet Disruption at ISP Office	Medium	Temporary inability to log bills	Enable offline caching / mobile responsive access via cellular network.